Statistical Analysis of Network Data – Assignment 2

Alex Gould

2026-10-15

1.

```
#Call igraph
library(igraph)
```

```
##
## Attaching package: 'igraph'
```

```
## The following objects are masked from 'package:stats':
##
##     decompose, spectrum
```

```
## The following object is masked from 'package:base':
##
##     union
```

```
#Read in Data
dolphins <- igraph::read_graph("/Users/alexgould/Desktop/dolphins.gml", "gml")
#Vertex Indices
V(dolphins)
```

```
## + 62/62 vertices, from a0aace5:
##  [1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
## [26] 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50
## [51] 51 52 53 54 55 56 57 58 59 60 61 62
```

```
#Vertex Names
V(dolphins)$label
```

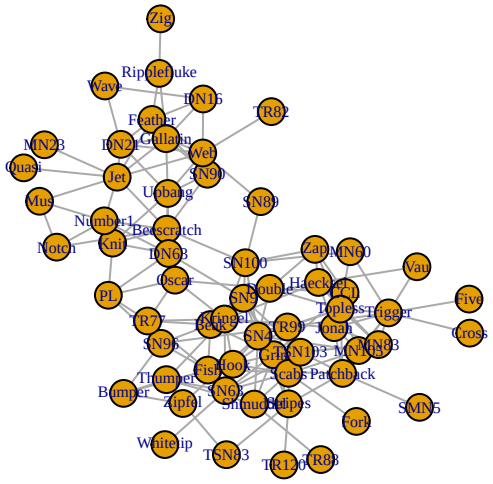
```
##  [1] "Beak"        "Beescratch"  "Bumper"      "CCL"         "Cross"
##  [6] "DN16"        "DN21"        "DN63"        "Double"      "Feather"
## [11] "Fish"        "Five"        "Fork"        "Gallatin"    "Grin"
## [16] "Haecksel"    "Hook"        "Jet"         "Jonah"       "Knit"
## [21] "Kringel"     "MN105"       "MN23"        "MN60"        "MN83"
## [26] "Mus"         "Notch"       "Number1"     "Oscar"       "Patchback"
## [31] "PL"          "Quasi"       "Ripplefluke" "Scabs"       "Shmuddel"
## [36] "SMN5"        "SN100"       "SN4"         "SN63"        "SN89"
## [41] "SN9"         "SN90"        "SN96"        "Stripes"     "Thumper"
## [46] "Topless"     "TR120"       "TR77"        "TR82"        "TR88"
## [51] "TR99"        "Trigger"     "TSN103"      "TSN83"       "Upbang"
## [56] "Vau"         "Wave"        "Web"         "Whitetip"    "Zap"
## [61] "Zig"         "Zipfel"
```

```
#Edge Indices
E(dolphins)
```

```
## + 159/159 edges from a0aace5:
```

```
##   [1]  4-- 9  6--10  7--10  1--11  3--11  6--14  7--14 10--14  1--15  4--15
##  [11]  1--16 15--17  2--18  7--18 10--18 14--18 16--19  2--20  8--20  9--21
##  [21] 17--21 19--21 19--22 18--23 15--25 16--25 19--25 18--26  2--27 26--27
##  [31]  2--28  8--28 18--28 26--28 27--28  2--29  9--29 21--29 11--30 19--30
##  [41] 22--30 25--30  8--31 20--31 29--31 18--32 10--33 14--33 13--34 15--34
##  [51] 17--34 22--34 15--35 34--35 30--36  2--37 21--37 24--37  9--38 15--38
##  [61] 17--38 22--38 34--38 35--38 37--38 15--39 17--39 21--39 34--39 37--40
##  [71]  1--41  8--41 15--41 16--41 34--41 37--41 38--41  2--42 10--42 14--42
##  [81]  1--43  3--43 11--43 31--43 15--44 30--44 34--44 38--44 39--44  3--45
##  [91] 21--45 35--45 39--45  9--46 16--46 19--46 22--46 24--46 25--46 30--46
## + ... omitted several edges
```

```
#Basic Graph Plot
plot(dolphins, vertex.size = 12, vertex.label.cex = 0.5,
     layout = layout.kamada.kawai)
```



a.

```
# Nodes with labels
cbind(V(dolphins)$label, V(dolphins))
```

```
##       [,1]          [,2]
##  [1,] "Beak"        "1"
##  [2,] "Beescratch"  "2"
##  [3,] "Bumper"      "3"
##  [4,] "CCL"         "4"
##  [5,] "Cross"       "5"
##  [6,] "DN16"        "6"
##  [7,] "DN21"        "7"
##  [8,] "DN63"        "8"
##  [9,] "Double"      "9"
## [10,] "Feather"     "10"
## [11,] "Fish"        "11"
## [12,] "Five"        "12"
## [13,] "Fork"        "13"
## [14,] "Gallatin"    "14"
## [15,] "Grin"        "15"
## [16,] "Haecksel"    "16"
## [17,] "Hook"        "17"
## [18,] "Jet"         "18"
```

```
## [19,] "Jonah"       "19"
## [20,] "Knit"        "20"
## [21,] "Kringel"     "21"
## [22,] "MN105"       "22"
## [23,] "MN23"        "23"
## [24,] "MN60"        "24"
## [25,] "MN83"        "25"
## [26,] "Mus"         "26"
## [27,] "Notch"       "27"
## [28,] "Number1"     "28"
## [29,] "Oscar"       "29"
## [30,] "Patchback"   "30"
## [31,] "PL"          "31"
## [32,] "Quasi"       "32"
## [33,] "Ripplefluke" "33"
## [34,] "Scabs"       "34"
## [35,] "Shmuddel"    "35"
## [36,] "SMN5"        "36"
## [37,] "SN100"       "37"
## [38,] "SN4"         "38"
## [39,] "SN63"        "39"
## [40,] "SN89"        "40"
## [41,] "SN9"         "41"
## [42,] "SN90"        "42"
## [43,] "SN96"        "43"
## [44,] "Stripes"     "44"
## [45,] "Thumper"     "45"
## [46,] "Topless"     "46"
## [47,] "TR120"       "47"
## [48,] "TR77"        "48"
## [49,] "TR82"        "49"
## [50,] "TR88"        "50"
## [51,] "TR99"        "51"
## [52,] "Trigger"     "52"
## [53,] "TSN103"      "53"
## [54,] "TSN83"       "54"
## [55,] "Upbang"      "55"
## [56,] "Vau"         "56"
## [57,] "Wave"        "57"
## [58,] "Web"         "58"
## [59,] "Whitetip"    "59"
## [60,] "Zap"         "60"
## [61,] "Zig"         "61"
## [62,] "Zipfel"      "62"
```

```
#Edges
E(dolphins)
```

```
## + 159/159 edges from a0aace5:
##   [1]  4-- 9  6--10  7--10  1--11  3--11  6--14  7--14 10--14  1--15  4--15
##  [11]  1--16 15--17  2--18  7--18 10--18 14--18 16--19  2--20  8--20  9--21
##  [21] 17--21 19--21 19--22 18--23 15--25 16--25 19--25 18--26  2--27 26--27
##  [31]  2--28  8--28 18--28 26--28 27--28  2--29  9--29 21--29 11--30 19--30
##  [41] 22--30 25--30  8--31 20--31 29--31 18--32 10--33 14--33 13--34 15--34
##  [51] 17--34 22--34 15--35 34--35 30--36  2--37 21--37 24--37  9--38 15--38
```

```
##  [61] 17--38 22--38 34--38 35--38 37--38 15--39 17--39 21--39 34--39 37--40
##  [71]  1--41  8--41 15--41 16--41 34--41 37--41 38--41  2--42 10--42 14--42
##  [81]  1--43  3--43 11--43 31--43 15--44 30--44 34--44 38--44 39--44  3--45
##  [91] 21--45 35--45 39--45  9--46 16--46 19--46 22--46 24--46 25--46 30--46
## + ... omitted several edges
```

```
# Node attributes
vertex_attr(dolphins)
```

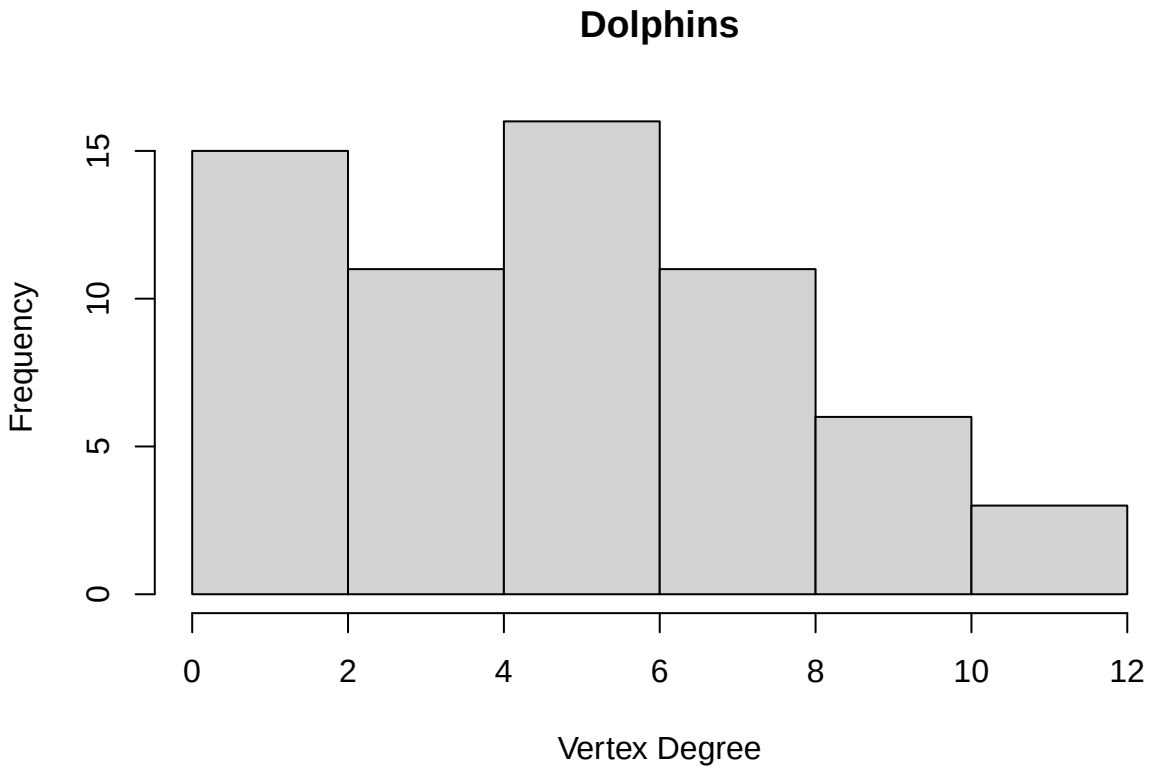
```
## $id
##  [1]  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
## [26] 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49
## [51] 50 51 52 53 54 55 56 57 58 59 60 61
##
## $label
##  [1] "Beak"        "Beescratch"  "Bumper"      "CCL"         "Cross"
##  [6] "DN16"        "DN21"        "DN63"        "Double"      "Feather"
## [11] "Fish"        "Five"        "Fork"        "Gallatin"    "Grin"
## [16] "Haecksel"    "Hook"        "Jet"         "Jonah"       "Knit"
## [21] "Kringel"     "MN105"       "MN23"        "MN60"        "MN83"
## [26] "Mus"         "Notch"       "Number1"     "Oscar"       "Patchback"
## [31] "PL"          "Quasi"       "Ripplefluke" "Scabs"       "Shmuddel"
## [36] "SMN5"        "SN100"       "SN4"         "SN63"        "SN89"
## [41] "SN9"         "SN90"        "SN96"        "Stripes"     "Thumper"
## [46] "Topless"     "TR120"       "TR77"        "TR82"        "TR88"
## [51] "TR99"        "Trigger"     "TSN103"      "TSN83"       "Upbang"
## [56] "Vau"         "Wave"        "Web"         "Whitetip"    "Zap"
## [61] "Zig"         "Zipfel"
```

```
# Edge attributes
edge_attr(dolphins)
```

```
## named list()
```

b.

```
library(igraph)
#A : Plot Degree Distribution
hist(igraph::degree(dolphins), xlab = "Vertex Degree",
     ylab = "Frequency", main = "Dolphins")
```




```
#B: Density
edge_density(dolphins)
```

```
## [1] 0.0840825
```

```
#C : Clustering
transitivity(dolphins, "global")
```

```
## [1] 0.3087757
```

```
#D: Centrality
cent_table<- data.frame(
closeness = igraph::closeness(dolphins),
harmonic = harmonic_centrality(dolphins, normalized = TRUE),
betweeness = igraph::betweenness(dolphins, normalized = TRUE),
eigenvector = eigen_centrality(dolphins)$vector,
pagerank = page_rank(dolphins)$vector
)
head(cent_table, n=10)
```

```
##      closeness  harmonic  betweeness eigenvector    pagerank
## 1  0.005681818 0.4259563 0.019082596  0.40693637 0.016965392
## 2  0.006097561 0.4486339 0.213324436  0.13324392 0.024650717
## 3  0.004629630 0.3504294 0.009072812  0.12590018 0.013338080
## 4  0.005050505 0.3687354 0.002373797  0.25123200 0.009629062
## 5  0.004081633 0.2936183 0.000000000  0.09274536 0.005079800
## 6  0.003906250 0.3056206 0.004380300  0.02077141 0.014428048
## 7  0.004385965 0.3568306 0.029372537  0.03860646 0.020053629
## 8  0.005988024 0.4218579 0.118238619  0.13585598 0.015643030
## 9  0.005952381 0.4352459 0.022365738  0.45316627 0.017098301
## 10 0.004132231 0.3515222 0.020894380  0.03862586 0.023458479
```

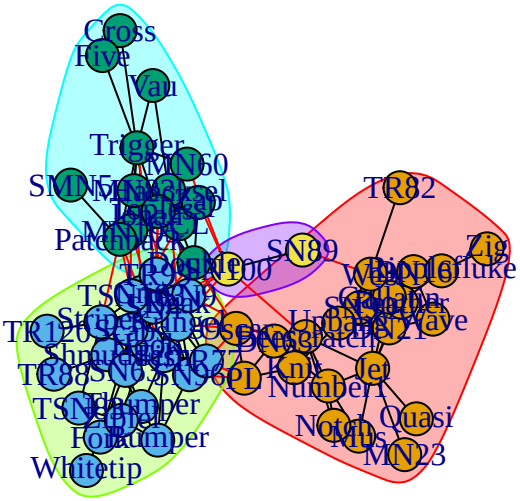
```
#Mean centrality across nodes
colMeans(cent_table)
```

```
##   closeness    harmonic  betweeness eigenvector    pagerank
## 0.005036721 0.379214198 0.039282567 0.287455449 0.016129032
```

```
#E: Assortativity
assortativity_degree(dolphins)
```

```
## [1] -0.04359403
```

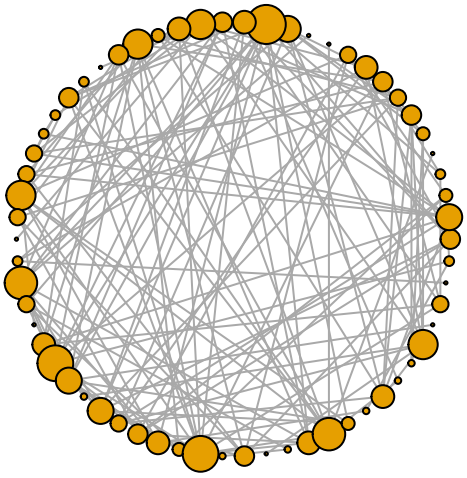
```
#F: Community Detection
plot(cluster_fast_greedy(dolphins), dolphins)
```



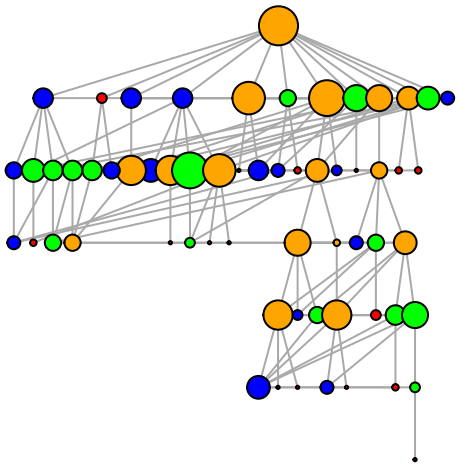
By the histogram we plotted, it seems that the degree distribution of the dolphins graph is fairly evenly distributed with some minor skewness to the right. We also see that the edge density is only 0.08 providing for a less dense network. Looking at the aggregated centrality, it seems like the network as a whole has substantially low closeness centrality with only 0.005 but quite high harmonic centrality with 0.379.There also seems to be negative assortativity in the network and 4 detected networks via fast and greedy.

c.

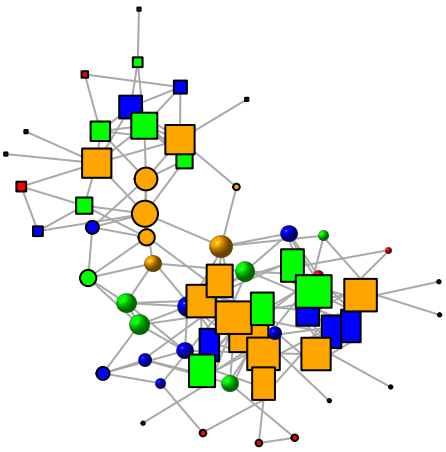
```
#Size of node correlates with degree
V(dolphins)$size <- 1.5*(igraph::degree(dolphins))
plot(dolphins, vertex.label = NA, layout = layout.circle)
```



```r
# Node color represented by Betweenness quantiles
b <- igraph::betweenness(dolphins, normalized = TRUE)
V(dolphins)$color <- ifelse(b < quantile(b, 0.25), "red",
                            ifelse(b < quantile(b, 0.5), "blue",
                                   ifelse(b < quantile(b, 0.75),
                                          "green", "orange")))
plot(dolphins, vertex.label = NA, layout = layout_as_tree)
```



```r
# Shape of nodes represented by eigenvector centrality quantiles
e <- eigen_centrality(dolphins)$vector
plot(dolphins, vertex.label = NA, vertex.shape = ifelse(e<quantile(e,0.25),
                  "square", ifelse(e < quantile(e, 0.5), "circle",
                    ifelse(e < quantile(e, 0.75), "sphere", "crectangle"))),
                   layout = layout_components)
```



Of these 3 graphs I belueve the tree layout represents the network the best. Also, with full disclosure I believe the different shapes of the nodes make the graph more confusing, so I believe the optimal graph for this network would be the second one when taking visualization into account.

d.

```r
# Simulation (Classical)
set.seed(1000)
dolphinsg <- vector('list', 1000)
for(i in 1:1000){
        dolphinsg[[i]] <- erdos.renyi.game(n = gorder(dolphins),
                                           p.or.m = edge_density(dolphins),
```

```
                                        type = "gnp")
}
```

```
## Warning: `erdos.renyi.game()` was deprecated in igraph 0.8.0.
## i Please use `sample_gnp()` instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
# Average Degree
avgdegree <- rep(mean(degree(dolphins)),1000)
#Observed Value
avgdegree[1]
```

```
## [1] 5.129032
```

```
#Simulation
avgdegreesim <- unlist(sapply(dolphinsg, function(x) mean(degree(x))))
# P Value
mean(avgdegree>avgdegreesim)
```

```
## [1] 0.464
```

```
# Centrality Measures

# Closeness
closeness <- rep(igraph::closeness(dolphins),1000)
#Observed Value
closeness[1]
```

```
## [1] 0.005681818
```

```
#Simulation
closenesssim <- unlist(lapply(dolphinsg, function(x) igraph::closeness(x)))
closenesssim[is.na(closenesssim)]<- 0
# P Value
mean(closeness>closenesssim)
```

```
## [1] 0.127629
```

```
#Harmonic
harmonic <- rep(igraph::harmonic_centrality(dolphins, normalized = TRUE),1000)
#Observed Value
harmonic[1]
```

```
## [1] 0.4259563
```

```
#Simulation
harmonicsim <- unlist(lapply(dolphinsg,
                function(x) igraph::harmonic_centrality(x, normalized = TRUE)))
# P Value
mean(harmonic>harmonicsim)
```

```
## [1] 0.2872903
```

```
#Betweenness
betweenness <- rep(igraph::betweenness(dolphins, normalized = TRUE),1000)
#Observed Value
betweenness[1]
```

```
## [1] 0.0190826
```

```
#Simulation
betweennesssim <- unlist(lapply(dolphinsg,
                      function(x) igraph::betweenness(x, normalized = TRUE)))
# P Value
mean(betweenness>betweennesssim)
```

```
## [1] 0.4914677
```

```
#Eigenvector
eigen <- rep(igraph::eigen_centrality(dolphins)$vector,1000)
#Observed Value
eigen[1]
```

```
## [1] 0.4069364
```

```
#Simulation
eigensim <- unlist(lapply(dolphinsg, function(x) igraph::eigen_centrality(x)$vector))
# P Value
mean(eigen>eigensim)
```

```
## [1] 0.3157903
```

```
#Page Rank
page_rank <- rep(igraph::page_rank(dolphins)$vector,1000)
#Observed Value
page_rank[1]
```

```
## [1] 0.01696539
```

```
#Simulation
page_ranksim <- unlist(lapply(dolphinsg, function(x) igraph::page_rank(x)$vector))
# P Value
mean(page_rank>page_ranksim)
```

```
## [1] 0.4845
```

```
#Assortativity
assortativity <- rep(igraph::assortativity_degree(dolphins),1000)
#Observed Value
assortativity[1]
```

```
## [1] -0.04359403
```

```
#Simulation
assortativitysim <- unlist(lapply(dolphinsg,
                                  function(x) igraph::assortativity_degree(x)))
# P Value
mean(assortativity>assortativitysim)
```

```
## [1] 0.482
```

```
# Clustering Coefficients
transitivity <- rep(igraph::transitivity(dolphins),1000)
#Observed Value
transitivity[1]
```

```
## [1] 0.3087757
```

```r
#Simulation
transitivitysim <- unlist(lapply(dolphinsg,
                                 function(x) igraph::transitivity(x)))
# P Value
mean(transitivity>transitivitysim)
```

```
## [1] 1
```

```r
#Density
edge_density <- rep(igraph::edge_density(dolphins),1000)
#Observed Value
edge_density[1]
```

```
## [1] 0.0840825
```

```r
#Simulation
edge_densitysim <- unlist(lapply(dolphinsg,
                                 function(x) igraph::edge_density(x)))
# P Value
mean(edge_density>edge_densitysim)
```

```
## [1] 0.464
```

```r
#Number of Communities
communities <- rep(length(cluster_fast_greedy(dolphins)),1000)
#Observed Value
communities[1]
```

```
## [1] 4
```

```r
#Simulation
communitiessim <- unlist(lapply(dolphinsg,
                                function(x)length(cluster_fast_greedy(x))))
# P Value
mean(communities>communitiessim)
```

```
## [1] 0
```

```r
summary(communitiessim)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   4.000   6.000   6.000   6.399   7.000  11.000
```

```r
#Simulation (Generalized)
set.seed(1000)
generalized_sim <- vector('list',1000)

for(i in 1:1000){
  generalized_sim[[i]]<-sample_degseq(out.deg = degree(dolphins),
                                      method = "configuration")
}


# Average Degree
avgdegree <- rep(mean(degree(dolphins)),1000)
#Observed Value
avgdegree[1]
```

```
## [1] 5.129032
```

```r
#Simulation
avgdegreesim <- unlist(sapply(generalized_sim, function(x) mean(degree(x))))
# P Value
mean(avgdegree>avgdegreesim)
```

```
## [1] 0
```

```r
# Centrality Measures

# Closeness
closeness <- rep(igraph::closeness(dolphins),1000)
#Observed Value
closeness[1]
```

```
## [1] 0.005681818
```

```r
#Simulation
closenesssim <- unlist(lapply(generalized_sim, function(x) igraph::closeness(x)))
closenesssim[is.na(closenesssim)]<- 0
# P Value
mean(closeness>closenesssim)
```

```
## [1] 0.06824194
```

```r
#Harmonic
harmonic <- rep(igraph::harmonic_centrality(dolphins, normalized = TRUE),1000)
#Observed Value
harmonic[1]
```

```
## [1] 0.4259563
```

```r
#Simulation
harmonicsim <- unlist(lapply(generalized_sim,
                function(x) igraph::harmonic_centrality(x, normalized = TRUE)))
# P Value
mean(harmonic>harmonicsim)
```

```
## [1] 0.1647258
```

```r
#Betweenness
betweenness <- rep(igraph::betweenness(dolphins, normalized = TRUE),1000)
#Observed Value
betweenness[1]
```

```
## [1] 0.0190826
```

```r
#Simulation
betweennesssim <- unlist(lapply(generalized_sim,
                        function(x) igraph::betweenness(x, normalized = TRUE)))
# P Value
mean(betweenness>betweennesssim)
```

```
## [1] 0.4325323
```

```r
#Eigenvector
eigen <- rep(igraph::eigen_centrality(dolphins)$vector,1000)
#Observed Value
eigen[1]
```

```
## [1] 0.4069364
```

```r
#Simulation
eigensim <- unlist(lapply(generalized_sim,
                          function(x) igraph::eigen_centrality(x)$vector))
# P Value
mean(eigen>eigensim)
```

```
## [1] 0.4110161
```

```r
#Page Rank
page_rank <- rep(igraph::page_rank(dolphins)$vector,1000)
#Observed Value
page_rank[1]
```

```
## [1] 0.01696539
```

```r
#Simulation
page_ranksim <- unlist(lapply(generalized_sim,
                              function(x) igraph::page_rank(x)$vector))
# P Value
mean(page_rank>page_ranksim)
```

```
## [1] 0.4806774
```

```r
#Assortativity
assortativity <- rep(igraph::assortativity_degree(dolphins),1000)
#Observed Value
assortativity[1]
```

```
## [1] -0.04359403
```

```r
#Simulation
assortativitysim <- unlist(lapply(generalized_sim,
                                  function(x) igraph::assortativity_degree(x)))
# P Value
mean(assortativity>assortativitysim)
```

```
## [1] 0.291
```

```r
# Clustering Coefficients
transitivity <- rep(igraph::transitivity(dolphins),1000)
#Observed Value
transitivity[1]
```

```
## [1] 0.3087757
```

```r
#Simulation
transitivitysim <- unlist(lapply(generalized_sim,
                                 function(x) igraph::transitivity(x)))
# P Value
mean(transitivity>transitivitysim)
```

```
## [1] 1
```

```r
#Density
edge_density <- rep(igraph::edge_density(dolphins),1000)
#Observed Value
edge_density[1]
```

```
## [1] 0.0840825
```

```
#Simulation
edge_densitysim <- unlist(lapply(generalized_sim,
                                 function(x) igraph::edge_density(x)))
# P Value
mean(edge_density>edge_densitysim)
```

```
## [1] 0
```

```
#Number of Communities
communities <- rep(length(cluster_fast_greedy(dolphins)),1000)
#Observed Value
communities[1]
```

```
## [1] 4
```

```
#Simulation
# communitiessim <- unlist(lapply(generalized_sim,
# function(x)length(cluster_fast_greedy(x))))
# Does not work: Error: Error in cluster_fast_greedy(x) :
 # Fast greedy community detection works only on graphs without multi-edges.
#Invalid value Source: community/fast_modularity.c:666
# P Value
#mean(communities>communitiessim)
#summary(communitiessim)
```

When observing the classical vs. the generalized random graph simulation, the first aspect I notice is that there are more significant results among the generalized random graph than the classical Erdos Renyi random graph for a given characteristic. Another complication worth mentioning is that the generalized random graph models did not work for simulating community detection via fast and greedy. This is why I commented it out at the end with the Error message included.

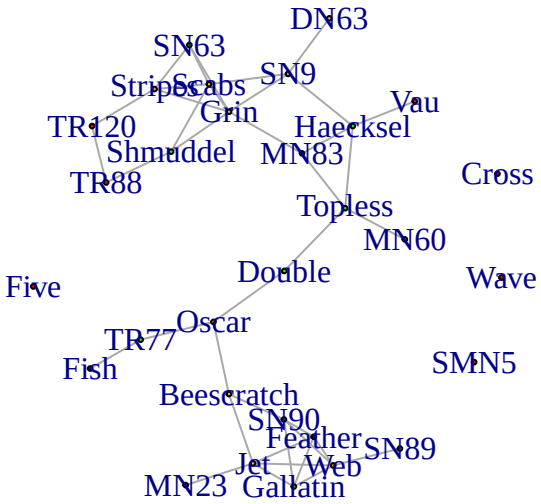
e.

```
#Call sand package
library(sand)
```

```
## Loading required package: igraphdata
```

```
##
## Statistical Analysis of Network Data with R, 2nd Edition
## Type in C2 (+ENTER) to start with Chapter 2.
```

```
#### Induced Subgraph Sampling ####
g1 <- igraph::induced_subgraph(dolphins, sample(seq_len(vcount(dolphins)) ,
                                                size = 30))
plot(g1, layout=layout.fruchterman.reingold,edge.arrow.size=.1, vertex.size=2)
```




```
mean(degree(g1))
```

```
## [1] 2.6
```

```
edge_density(g1)
```

```
## [1] 0.08965517
```

```
transitivity(g1, type=c("global"))
```

```
## [1] 0.3962264
```

```
eigen_centrality(g1)$vector
```

```
##  [1] 4.900871e-01 4.815876e-18 8.108259e-03 4.324123e-02 9.484583e-01
##  [6] 1.006794e-02 4.815876e-18 9.484583e-01 5.015946e-02 2.197266e-02
## [11] 9.206882e-01 2.340913e-01 6.203553e-03 2.454364e-02 1.456703e-01
## [16] 4.518409e-02 2.671445e-02 4.815876e-18 3.330407e-02 2.542569e-01
## [21] 3.189003e-02 8.611689e-01 3.564240e-02 2.439876e-02 1.153502e-02
## [26] 3.959752e-02 9.725189e-03 5.586698e-03 4.815876e-18 1.000000e+00
```

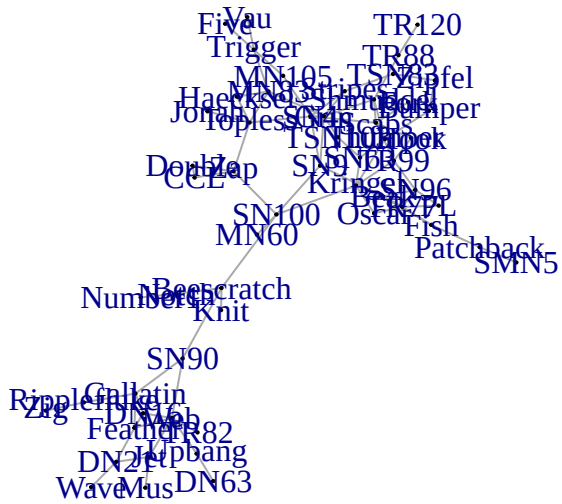
```
assortativity_degree(g1)
```

```
## [1] 0.02836336
```

```
#### Incident Subgraph Sampling ####
g2 <- igraph::subgraph.edges(dolphins, sample(seq_len(ecount(dolphins)) ,
                                              size = 80))
```

```
## Warning: `subgraph.edges()` was deprecated in igraph 2.1.0.
## i Please use `subgraph_from_edges()` instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
plot(g2, layout=layout.fruchterman.reingold, edge.arrow.size=.1, vertex.size=1)
```




```r
mean(degree(g2))
```

```
## [1] 2.807018
```

```r
edge_density(g2)
```

```
## [1] 0.05012531
```

```r
transitivity(g2, type=c("global"))
```

```
## [1] 0.1682243
```

```r
eigen_centrality(g2)$vector
```

```
##  [1] 0.740120458 0.205046936 0.158020140 0.125055602 0.025277258 0.011071517
##  [7] 0.002685017 0.125055602 0.027505918 0.453894799 0.061010735 0.178779123
## [13] 0.033202774 0.976951276 0.280065907 0.350472662 0.013633194 0.147636419
## [19] 0.051677862 0.944388823 0.313040164 0.160467037 0.525452477 0.003435966
## [25] 0.055183023 0.013907746 0.474590055 0.122153921 0.412697970 0.008935659
## [31] 0.709358125 0.604088711 0.030786383 0.636699604 1.000000000 0.467904862
## [37] 0.845249074 0.070022820 0.698813989 0.537401573 0.548013429 0.585790399
## [43] 0.040973605 0.938685142 0.009976881 0.162574687 0.681244013 0.242077822
## [49] 0.364146267 0.155345640 0.010653584 0.061010735 0.002790348 0.039586175
## [55] 0.371138840 0.002252049 0.078977398
```

```r
assortativity_degree(g2)
```

```
## [1] 0.01806156
```

```r
#### Snowball Sampling  ####
n_seeds=5
g3 <- graph.neighborhood(dolphins, order=1,
      nodes=sample(seq_len(vcount(dolphins)) ,size = n_seeds), mode=c("all"))
```

```
## Warning: `graph.neighborhood()` was deprecated in igraph 2.0.0.
## i Please use `make_ego_graph()` instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```r
g3 <- graph.disjoint.union(g3)
```

```
## Warning: `graph.disjoint.union()` was deprecated in igraph 2.0.0.
## i Please use `disjoint_union()` instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
mean(degree(g3))
```

```
## [1] 1.529412
```

```
mean(betweenness(g3))
```

```
## [1] 1.235294
```

```
transitivity(g3, type=c("global"))
```

```
## [1] 0.125
```

```
eigen_centrality(g3)$vector
```

```
##  [1] 0.0000000 0.0000000 0.0000000 0.0000000 0.3516911 0.3516911 0.3516911
##  [8] 1.0000000 0.5424745 0.3516911 0.5424745 0.3516911 0.0000000 0.0000000
## [15] 0.0000000 0.0000000 0.0000000
```

```
assortativity_degree(g3)
```

```
## [1] -0.3696639
```

From Induced Subgraph sampling to Incident subgraph sampling, we see that mean degree, edge density, and assortativity stay about the same, yet global transitivity splits in half from 0.3962 to 0.1682. More apparent is the drastic increase in eigenvector centrality in this same comparison. As for snowball sampling, using the same values in the sample code, we see a drastic increase across all measures except transitivity which stays at about 0.125. There is also negative assortativity which represents the true network better than the other sampling methods.

f.

```
# Simulation (Classical)
set.seed(1000)
dolphinsgmod <- vector('list', 1000)
for(i in 1:1000){
        dolphinsgmod[[i]] <- erdos.renyi.game(n = gorder(dolphins),
p.or.m = 0.042, type = "gnp") # Cut sampling p in half from 0.084 to 0.042
}

# Mean size
storeedge<- numeric(1000)
for(i in 1:1000){
storeedge[i] <- ecount(dolphinsgmod[[i]])
}
storeedgedolphinsg <- numeric(1000)
for(i in 1:1000){
storeedgedolphinsg[i] <- ecount(dolphinsg[[i]])
}
mean(storeedge)
```

```
## [1] 79.714
```

```
mean(storeedgedolphinsg) # About double the edges in the original, makes sense
```

```
## [1] 159.456
```

```
ecount(dolphins)
```

```
## [1] 159
```

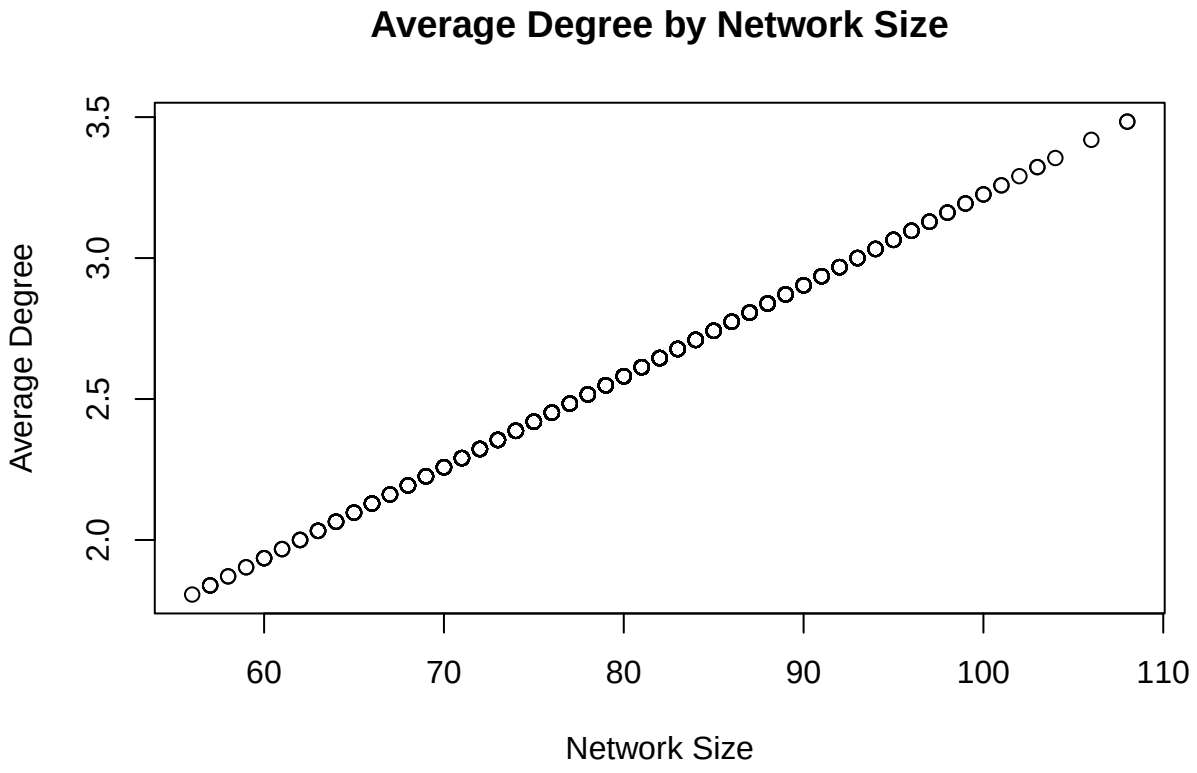
```
# Average Degree
avgdegree <- rep(mean(degree(dolphins)),1000)
#Observed Value
avgdegree[1]
```

```
## [1] 5.129032
```

```
#Simulation
avgdegreesim <- unlist(sapply(dolphinsgmod, function(x) mean(degree(x))))
# P Value
mean(avgdegree>avgdegreesim)
```

```
## [1] 1
```

```
plot(x=unlist(lapply(dolphinsgmod, function(x) ecount(x))),
     y=unlist(lapply(dolphinsgmod, function(x) (mean(degree((x)))))),
     main = "Average Degree by Network Size",
     xlab = "Network Size",
     ylab = "Average Degree")
```




```
# Centrality Measures

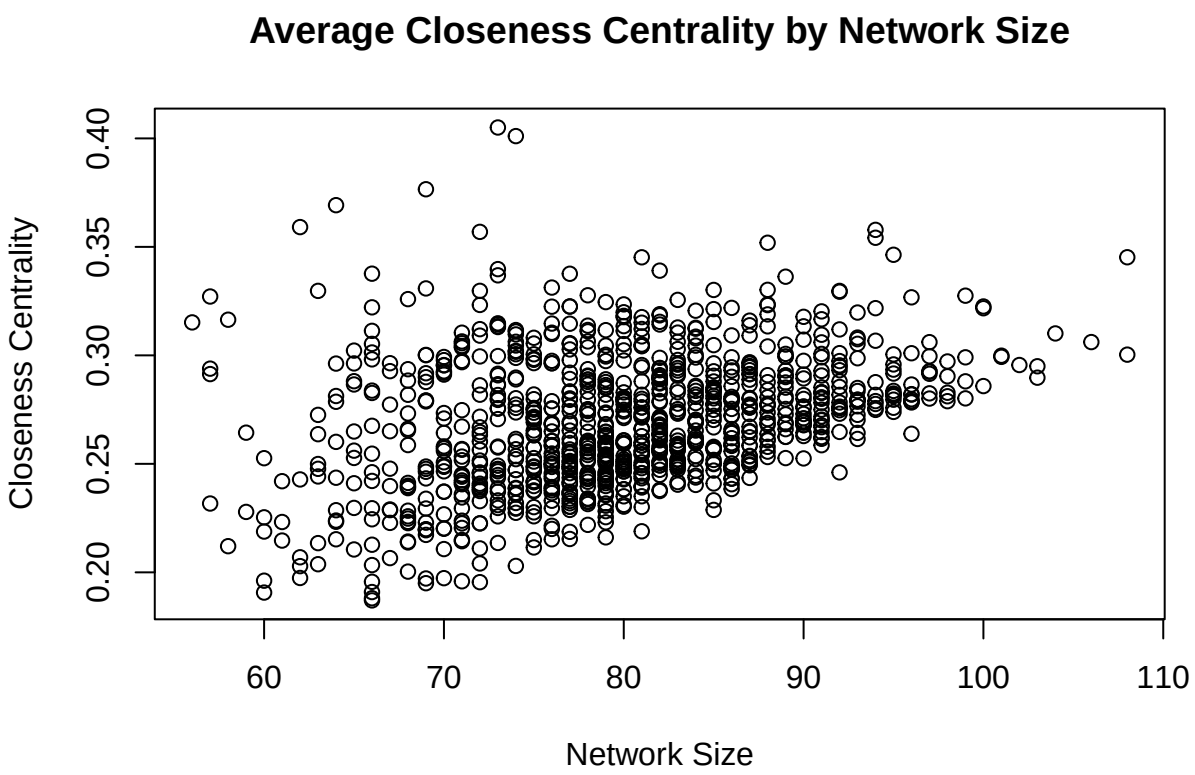
# Closeness
closeness <- rep(igraph::closeness(dolphins),1000)
#Observed Value
closeness[1]
```

```
## [1] 0.005681818
```

```
#Simulation
closenesssim <- unlist(lapply(dolphinsgmod, function(x) igraph::closeness(x)))
closenesssim[is.na(closenesssim)]<- 0
# P Value
mean(closeness>closenesssim)
```

```
## [1] 0.6536935
```

```
plot(x=unlist(lapply(dolphinsgmod, function(x) ecount(x))),
     y=unlist(lapply(dolphinsgmod, function(x) mean(closeness(x, normalized=TRUE),
                                                    na.rm=TRUE)) ),
     main = "Average Closeness Centrality by Network Size",
     xlab = "Network Size",
     ylab = "Closeness Centrality")
```



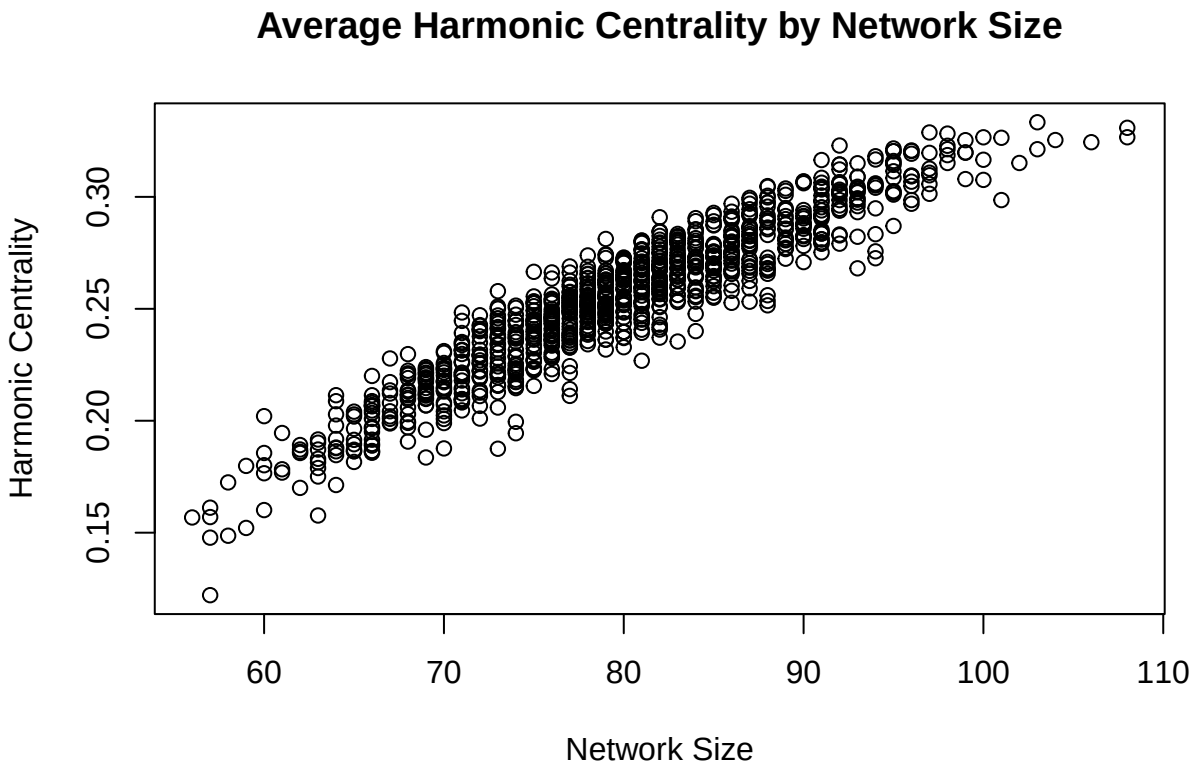

```
#Harmonic
harmonic <- rep(igraph::harmonic_centrality(dolphins, normalized = TRUE),1000)
#Observed Value
harmonic[1]
```

```
## [1] 0.4259563
```

```
#Simulation
harmonicsim <- unlist(lapply(dolphinsgmod,
                function(x) igraph::harmonic_centrality(x, normalized = TRUE)))
# P Value
mean(harmonic>harmonicsim)
```

```
## [1] 0.8780645
```

```
plot(x=unlist(lapply(dolphinsgmod, function(x) ecount(x))),
     y=unlist(lapply(dolphinsgmod,
          function(x) mean(igraph::harmonic_centrality(x, normalized = TRUE)))),
     main = "Average Harmonic Centrality by Network Size",
     xlab = "Network Size",
     ylab = "Harmonic Centrality")
```



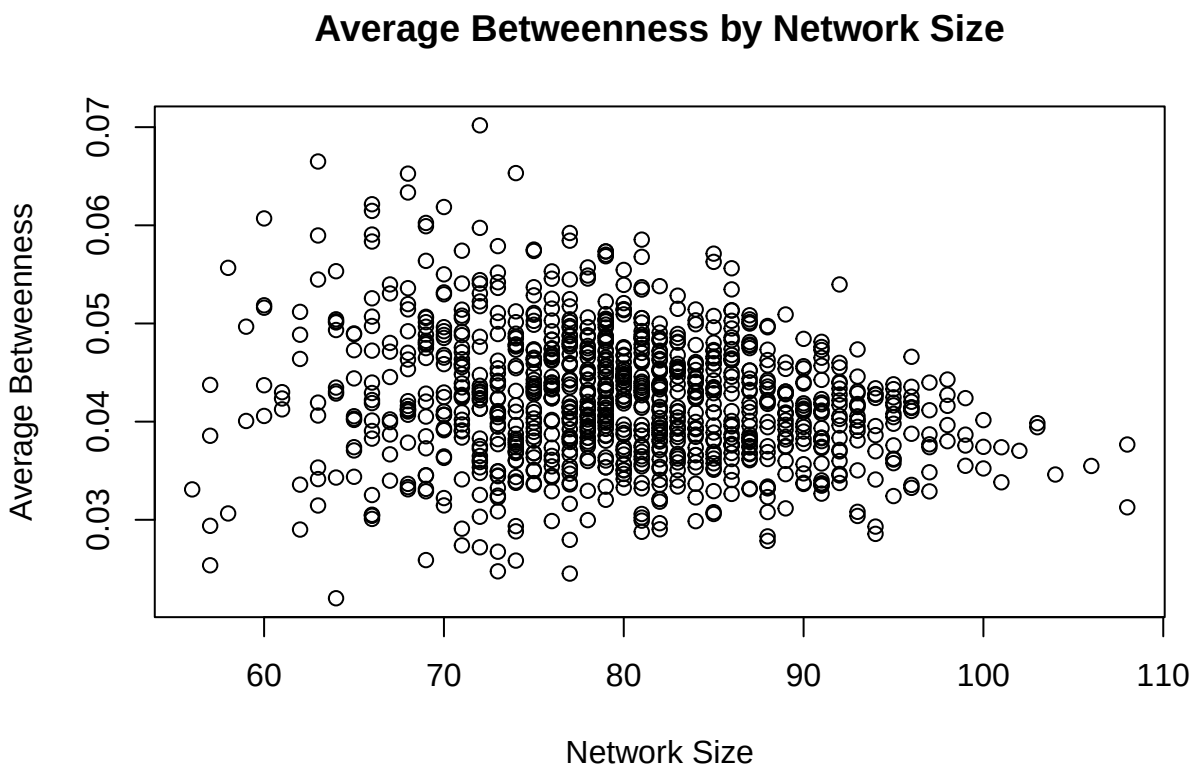

```
#Betweenness
betweenness <- rep(igraph::betweenness(dolphins, normalized = TRUE),1000)
#Observed Value
betweenness[1]
```

```
## [1] 0.0190826
```

```
#Simulation
betweennesssim <- unlist(lapply(dolphinsgmod,
                        function(x) igraph::betweenness(x, normalized = TRUE)))
# P Value
mean(betweenness>betweennesssim)
```

```
## [1] 0.4735806
```

```
plot(x=unlist(lapply(dolphinsgmod, function(x) ecount(x))),
     y=unlist(lapply(dolphinsgmod,
                    function(x) mean(betweenness(x, normalized=TRUE)))),
     main = "Average Betweenness by Network Size",
     xlab = "Network Size",
     ylab = "Average Betweenness")
```



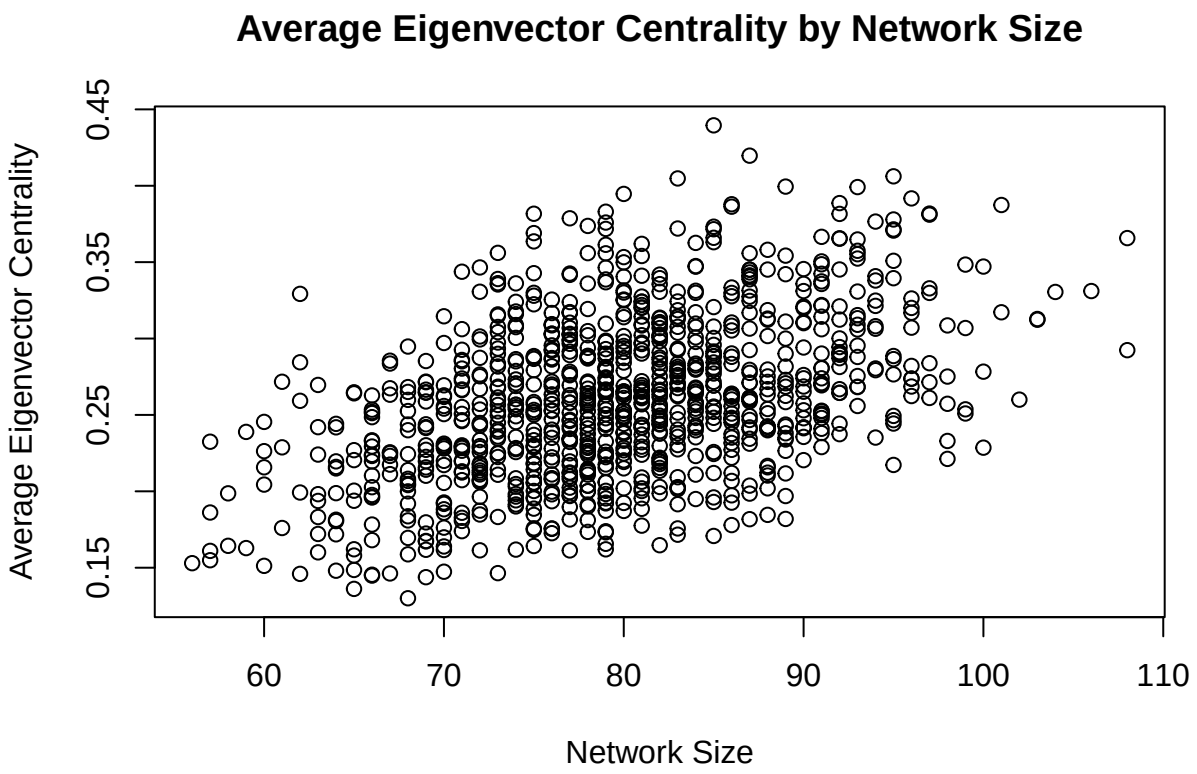

```
#Eigenvector
eigen <- rep(igraph::eigen_centrality(dolphins)$vector,1000)
#Observed Value
eigen[1]
```

```
## [1] 0.4069364
```

```
#Simulation
eigensim <- unlist(lapply(dolphinsgmod,
                          function(x) igraph::eigen_centrality(x)$vector))
# P Value
mean(eigen>eigensim)
```

```
## [1] 0.5091129
```

```
plot(x=unlist(lapply(dolphinsgmod, function(x) ecount(x))),
     y=unlist(lapply(dolphinsgmod,
                     function(x) mean(igraph::eigen_centrality((x))$vector))),
     main = "Average Eigenvector Centrality by Network Size",
     xlab = "Network Size",
     ylab = "Average Eigenvector Centrality")
```



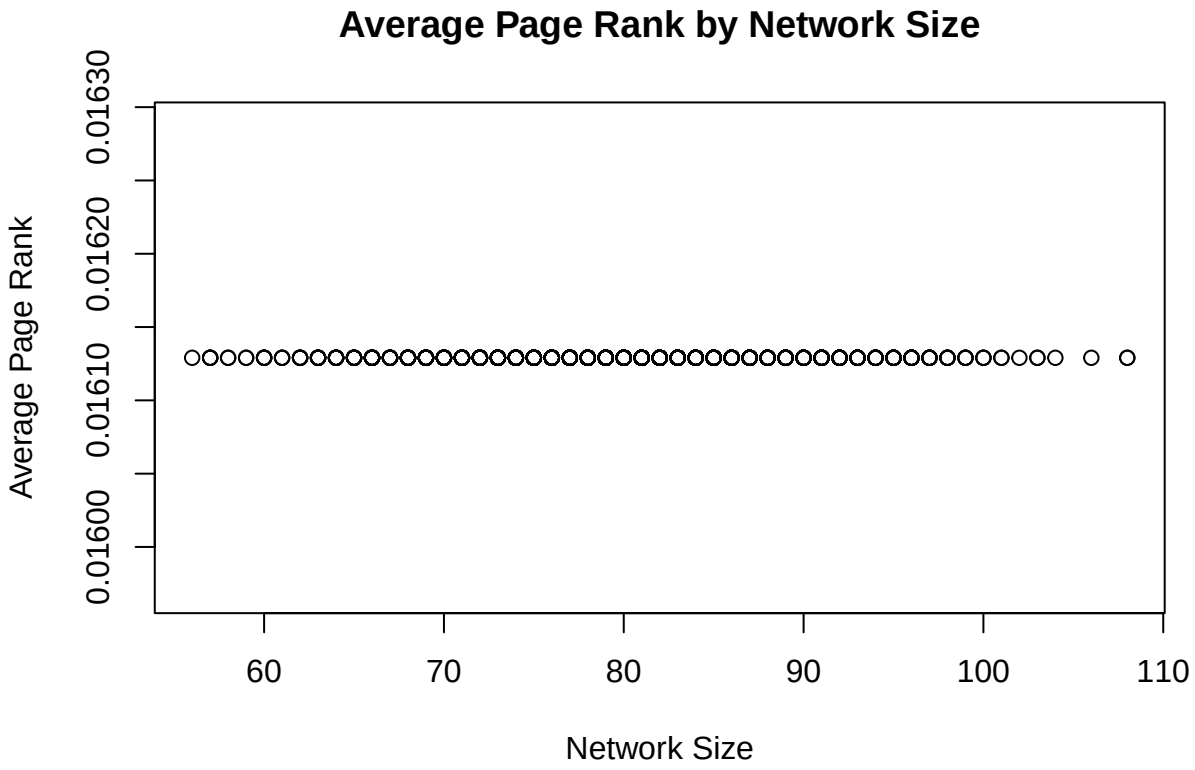

```
#Page Rank
page_rank <- rep(igraph::page_rank(dolphins)$vector,1000)
#Observed Value
page_rank[1]
```

```
## [1] 0.01696539
```

```
#Simulation
page_ranksim <- unlist(lapply(dolphinsgmod,
                              function(x) igraph::page_rank(x)$vector))
# P Value
mean(page_rank>page_ranksim)
```

```
## [1] 0.4954032
```

```
plot(x=unlist(lapply(dolphinsgmod, function(x) ecount(x))),
     y=unlist(lapply(dolphinsgmod, function(x) mean(page_rank((x))$vector))),
     main = "Average Page Rank by Network Size",
     xlab = "Network Size",
     ylab = "Average Page Rank")
```



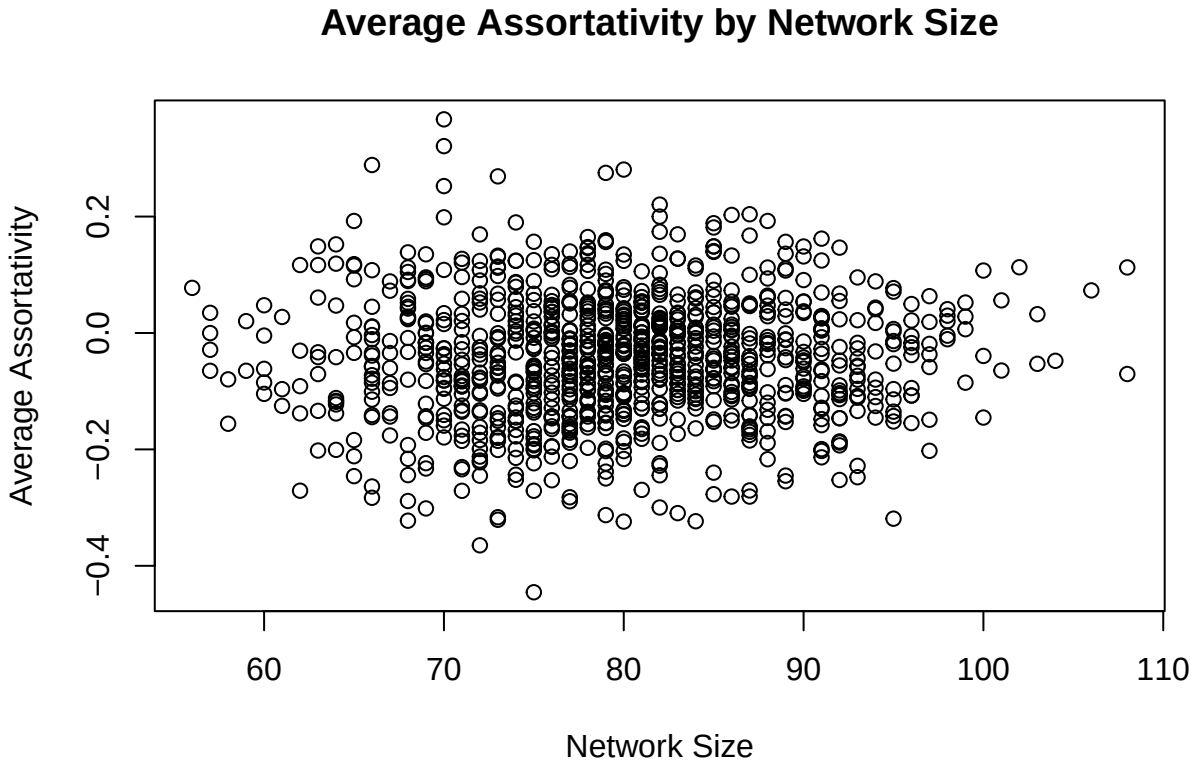

```
#Assortativity
assortativity <- rep(igraph::assortativity_degree(dolphins),1000)
#Observed Value
assortativity[1]
```

```
## [1] -0.04359403
```

```
#Simulation
assortativitysim <- unlist(lapply(dolphinsg,
                                  function(x) igraph::assortativity_degree(x)))
# P Value
mean(assortativity>assortativitysim)
```

```
## [1] 0.482
```

```
plot(x=unlist(lapply(dolphinsgmod, function(x) ecount(x))),
     y=unlist(lapply(dolphinsgmod, function(x) mean(assortativity_degree((x))))),
     main = "Average Assortativity by Network Size",
     xlab = "Network Size",
     ylab = "Average Assortativity")
```



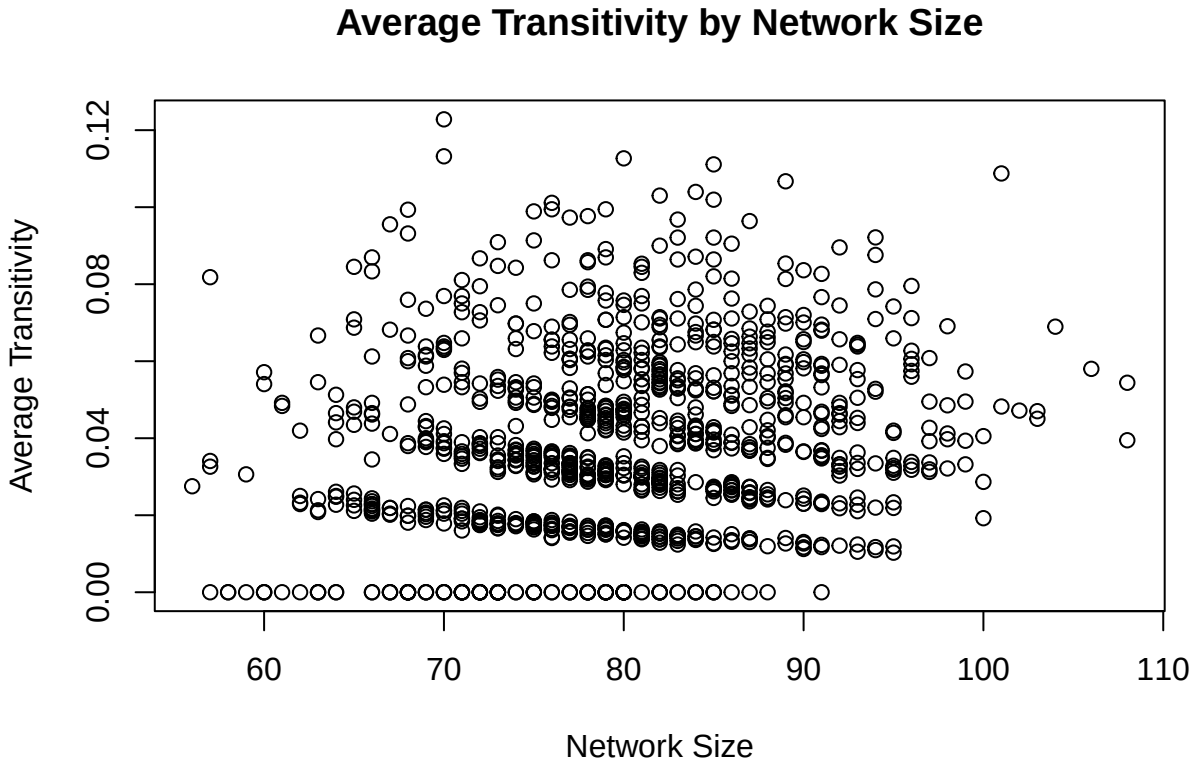

```
# Clustering Coefficients
transitivity <- rep(igraph::transitivity(dolphins),1000)
#Observed Value
transitivity[1]
```

```
## [1] 0.3087757
```

```
#Simulation
transitivitysim <- unlist(lapply(dolphinsgmod,
                                 function(x) igraph::transitivity(x)))
# P Value
mean(transitivity>transitivitysim)
```

```
## [1] 1
```

```
plot(x=unlist(lapply(dolphinsgmod, function(x) ecount(x))),
     y=unlist(lapply(dolphinsgmod, function(x) mean(transitivity((x))))),
     main = "Average Transitivity by Network Size",
     xlab = "Network Size",
     ylab = "Average Transitivity")
```



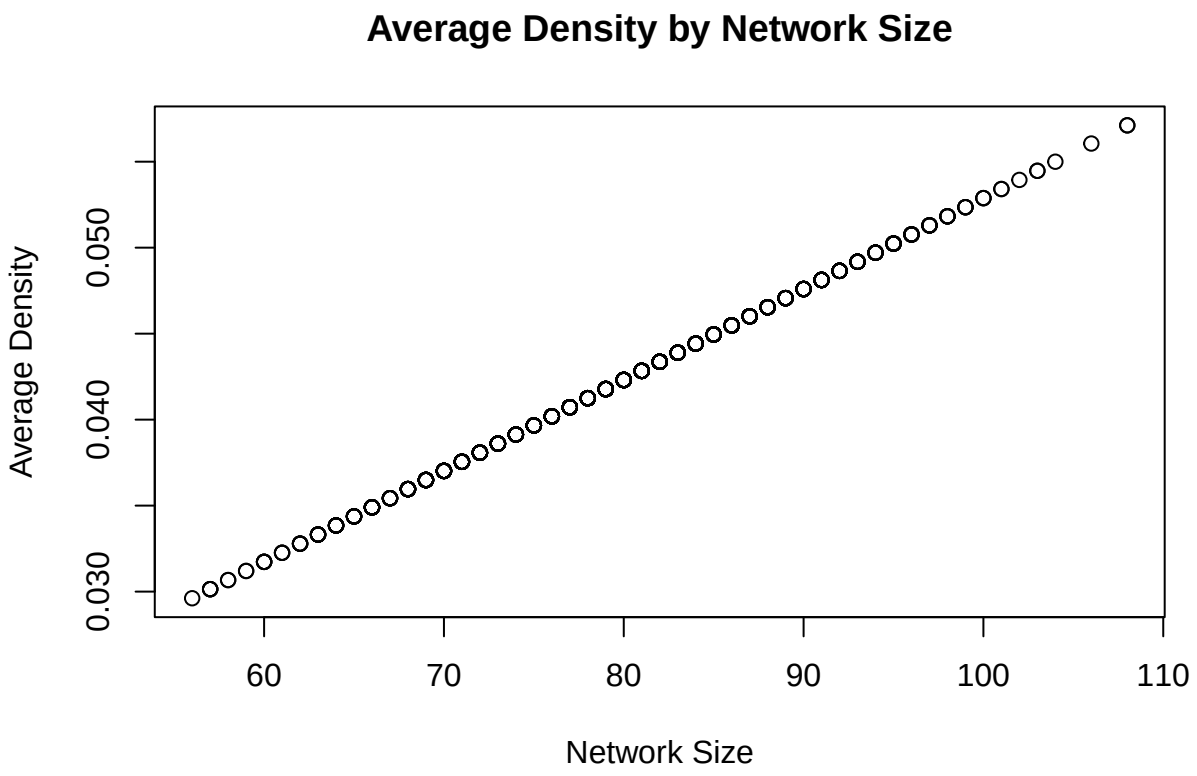

```
#Density
edge_density <- rep(igraph::edge_density(dolphins),1000)
#Observed Value
edge_density[1]
```

```
## [1] 0.0840825
```

```
#Simulation
edge_densitysim <- unlist(lapply(dolphinsgmod,
                                 function(x) igraph::edge_density(x)))
# P Value
mean(edge_density>edge_densitysim)
```

```
## [1] 1
```

```
plot(x=unlist(lapply(dolphinsgmod, function(x) ecount(x))),
     y=unlist(lapply(dolphinsgmod, function(x) mean(edge_density((x))))),
     main = "Average Density by Network Size",
     xlab = "Network Size",
     ylab = "Average Density")
```




```
#Number of Communities
communities <- rep(length(cluster_fast_greedy(dolphins)),1000)
#Observed Value
communities[1]
```

```
## [1] 4
```

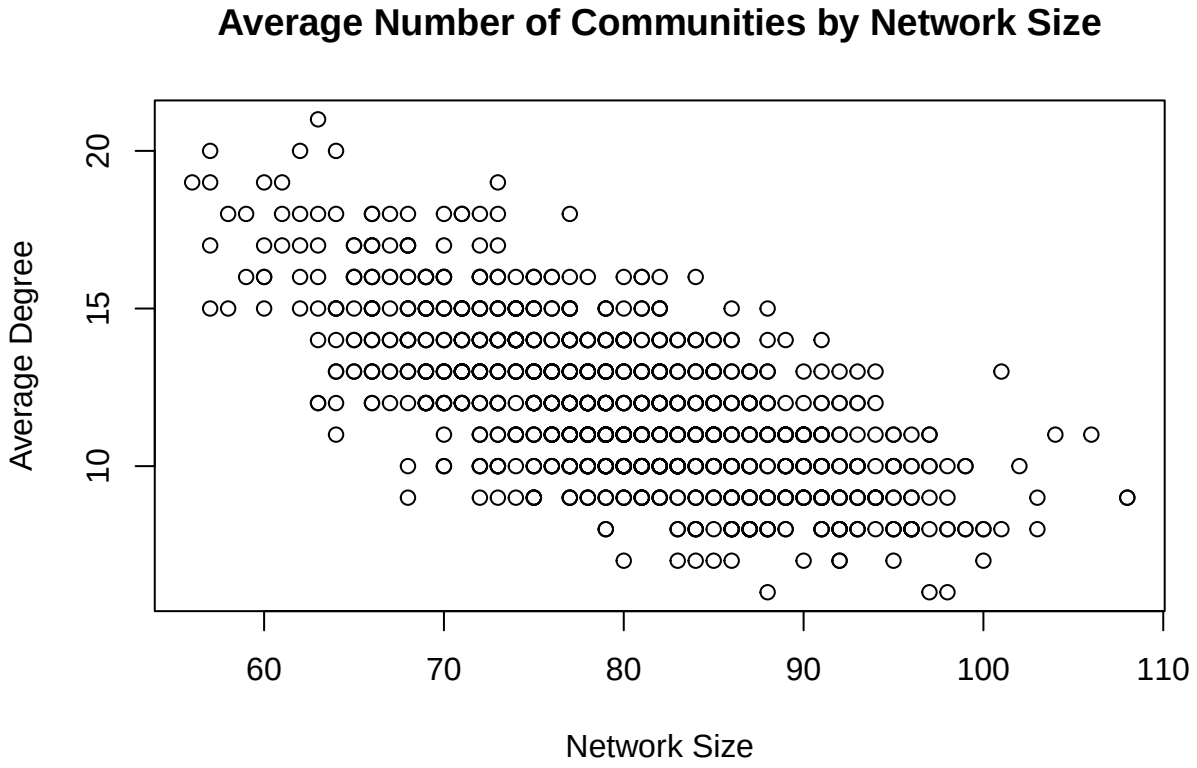
```
#Simulation
communitiessim <- unlist(lapply(dolphinsgmod,
                                function(x)length(cluster_fast_greedy(x))))
# P Value
mean(communities>communitiessim)
```

```
## [1] 0
```

```
summary(communitiessim)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    6.00   10.00   12.00   11.93   14.00   21.00
```

```
plot(x=unlist(lapply(dolphinsgmod, function(x) ecount(x))),
     y=unlist(lapply(dolphinsgmod,
                     function(x) mean(length(cluster_fast_greedy((x)))))),
     main = "Average Number of Communities by Network Size",
     xlab = "Network Size",
     ylab = "Average Degree")
```



I chose to re-do the erdos renyi and vary the sampling probabilities by cutting the edge density in half from 0.084 to 0.042. I will now review the plots I made for the mean of each characteristic against the network size vector. First, the Average degree seems to have an exactly linear relationship with the size of the network. This intuitively makes sense because the nodes are staying relatively constant while network size varies, especially in this formulation of the network. Average Closeness centrality seems to have a very slight positive relationship with network size. Average Harmonic Centrality has a stronger positive linear relationship with Network Size, Average Betweenness Centrality has a slight negative relationship with network size, Average Eigenvector Centrality seems to have a slight positive relationship with Network Size, Average Page Rank Centrality stays constant with Network Size, Average Assortativity stays approximately constant, Average Transitivity by Network Size seems to have an approximately constant relationship with Network Size, while it also has an interesting pattern where there are multiple strong relationshiips and a good amount of observations with average transitivity 0. Average Edge Density by Network Size seems to have a constant linear relationship with Network Size, The average number of communities seem to have a negative relationship with Network Size with very little overlapping points.